

# Explination of AI project (Predict Car Prices)

Prepared by:

روان رضا الوكيل

رودينا علي هدهد

حفصه نبيل يحيى

ساره محمد كمال

## 1-Using pandas:-

In Jupyter editor, We imported pandas, so we can analyze and organize data, we name it as pd so we can recall it easily.

## 2-loading data:-

We loaded the dataset (downloaded from kaggle) by using the function `pd.read_csv()` from pandas, and we put the path of the file of dataset as an input in the function.

We named the dataset as variable called `df`, so we can use it later easily.

As we see in this screenshot, after we recall `df`, we have the dataset presented in a table of 4340 rows x 8 columns. Each column is feature that describe the cars.

```
import needed library
```

```
[2]: import pandas as pd
```

```
loading data
```

```
[7]: df = pd.read_csv(r"C:\Users\DELL\Downloads\CAR DETAILS FROM CAR DEKHO.csv")
```

```
[8]: df
```

```
[8]:
```

|      | name                                | year | selling_price | km_driven | fuel   | seller_type | transmission | owner        |
|------|-------------------------------------|------|---------------|-----------|--------|-------------|--------------|--------------|
| 0    | Maruti 800 AC                       | 2007 | 60000         | 70000     | Petrol | Individual  | Manual       | First Owner  |
| 1    | Maruti Wagon R LXI Minor            | 2007 | 135000        | 50000     | Petrol | Individual  | Manual       | First Owner  |
| 2    | Hyundai Verna 1.6 SX                | 2012 | 600000        | 100000    | Diesel | Individual  | Manual       | First Owner  |
| 3    | Datsun RediGO T Option              | 2017 | 250000        | 46000     | Petrol | Individual  | Manual       | First Owner  |
| 4    | Honda Amaze VX i-DTEC               | 2014 | 450000        | 141000    | Diesel | Individual  | Manual       | Second Owner |
| ...  | ...                                 | ...  | ...           | ...       | ...    | ...         | ...          | ...          |
| 4335 | Hyundai i20 Magna 1.4 CRDi (Diesel) | 2014 | 409999        | 80000     | Diesel | Individual  | Manual       | Second Owner |
| 4336 | Hyundai i20 Magna 1.4 CRDi          | 2014 | 409999        | 80000     | Diesel | Individual  | Manual       | Second Owner |
| 4337 | Maruti 800 AC BSIII                 | 2009 | 110000        | 83000     | Petrol | Individual  | Manual       | Second Owner |
| 4338 | Hyundai Creta 1.6 CRDi SX Option    | 2016 | 865000        | 90000     | Diesel | Individual  | Manual       | First Owner  |
| 4339 | Renault KWID RXT                    | 2016 | 225000        | 40000     | Petrol | Individual  | Manual       | First Owner  |

4340 rows x 8 columns

### 3-Explore data:-

- **df.head():**

this function show first 5 rows, by checking them we noticed that there is no missing cells and nonrepeated columns

- **df.tail():**

this function shows last 5

rows, by checking them we noticed that there is no missing cells and the file readed correctly to the end.

- **df.sample(5):**

after checking a random 5 rows, we noticed that there is no missing cells

- **df.info():**

If rangeindex = non-null count ,there is no missing cells, and we have 2 types of data(int , object).

- **df.duplicated().sum():**

by using this function we deduced the number of repeated rows =763.

- **df.isnull().sum():**

This function explain the number of missing cells in each row , we notice that there is zero missing cells .

- **df.isna().sum():**

This function explain the number of missing cells in each row , we notice that there is zero missing cells.

- **dat.isna().sum().plot(kind = 'bar'):** In general, this function checks every

explore data

```
[9]: df.head()
```

|   | name                     | year | selling_price | km_driven | fuel   | seller_type | transmission | owner        |
|---|--------------------------|------|---------------|-----------|--------|-------------|--------------|--------------|
| 0 | Maruti 800 AC            | 2007 | 60000         | 70000     | Petrol | Individual  | Manual       | First Owner  |
| 1 | Maruti Wagon R LXI Minor | 2007 | 135000        | 50000     | Petrol | Individual  | Manual       | First Owner  |
| 2 | Hyundai Verna 1.6 SX     | 2012 | 600000        | 100000    | Diesel | Individual  | Manual       | First Owner  |
| 3 | Datsun RediGO T Option   | 2017 | 250000        | 46000     | Petrol | Individual  | Manual       | First Owner  |
| 4 | Honda Amaze VX i-DTEC    | 2014 | 450000        | 141000    | Diesel | Individual  | Manual       | Second Owner |

```
[12]: df.tail()
```

|      | name                                | year | selling_price | km_driven | fuel   | seller_type | transmission | owner        |
|------|-------------------------------------|------|---------------|-----------|--------|-------------|--------------|--------------|
| 4335 | Hyundai i20 Magna 1.4 CRDi (Diesel) | 2014 | 409999        | 80000     | Diesel | Individual  | Manual       | Second Owner |
| 4336 | Hyundai i20 Magna 1.4 CRDi          | 2014 | 409999        | 80000     | Diesel | Individual  | Manual       | Second Owner |
| 4337 | Maruti 800 AC BSIII                 | 2009 | 110000        | 83000     | Petrol | Individual  | Manual       | Second Owner |
| 4338 | Hyundai Creta 1.6 CRDi SX Option    | 2016 | 865000        | 90000     | Diesel | Individual  | Manual       | First Owner  |
| 4339 | Renault KWID RXT                    | 2016 | 225000        | 40000     | Petrol | Individual  | Manual       | First Owner  |

```
[14]: df.sample(5)
```

|      | name                          | year | selling_price | km_driven | fuel   | seller_type | transmission | owner        |
|------|-------------------------------|------|---------------|-----------|--------|-------------|--------------|--------------|
| 1951 | Hyundai Verna 1.6 SX CRDi (O) | 2013 | 315000        | 80000     | Diesel | Individual  | Manual       | Second Owner |
| 2817 | Hyundai Grand i10 Sportz      | 2016 | 375000        | 25000     | Petrol | Individual  | Manual       | First Owner  |
| 3965 | Maruti Swift VXI              | 2012 | 370000        | 59858     | Petrol | Dealer      | Manual       | First Owner  |
| 470  | Hyundai i10 Era 1.1           | 2009 | 120000        | 120000    | Petrol | Individual  | Manual       | Second Owner |
| 4225 | Hyundai i20 1.4 Asta Option   | 2017 | 780000        | 50000     | Diesel | Individual  | Manual       | First Owner  |

```
[15]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4340 entries, 0 to 4339
Data columns (total 8 columns):
#   Column              Non-Null Count  Dtype
---  --
0   name                 4340 non-null  object
1   year                 4340 non-null  int64
2   selling_price        4340 non-null  int64
3   km_driven            4340 non-null  int64
4   fuel                 4340 non-null  object
5   seller_type          4340 non-null  object
6   transmission         4340 non-null  object
7   owner                4340 non-null  object
dtypes: int64(3), object(5)
memory usage: 271.4+ KB
```

```
[20]: df.duplicated().sum()
```

```
[435]: dat.isna().sum().plot(kind = 'bar')
```

```
[20]: fig
```

```
[435]: <Axes: >
```

```
[23]: df
```

```
[23]:
```

```
name
```

```
year
```

```
selling_price
```

```
km_driven
```

```
fuel
```

```
seller_type
```

```
transmission
```

```
owner
```

```
[24]: df
```

```
[24]:
```

```
name
```

```
year
```

```
selling_price
```

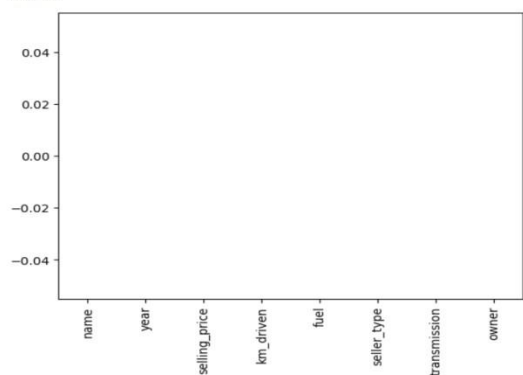
```
km_driven
```

```
fuel
```

```
seller_type
```

```
transmission
```

```
owner
```



single cell, If a cell is empty, it marks it as True; otherwise, it marks it as False.

Plot function takes those counts and turns them into a Bar Chart.

In this image, the database is currently clean because there is no missing values so there is nothing for the bar chart to display.

- **df.describe():**  
this function shows numerical statistics and some properties of each feature such that (min, max) etc...

```
[25]: df.describe()
[25]:
```

|       | year        | selling_price | km_driven     |
|-------|-------------|---------------|---------------|
| count | 4340.000000 | 4.340000e+03  | 4340.000000   |
| mean  | 2013.090783 | 5.041273e+05  | 66215.777419  |
| std   | 4.215344    | 5.785487e+05  | 46644.102194  |
| min   | 1992.000000 | 2.000000e+04  | 1.000000      |
| 25%   | 2011.000000 | 2.087498e+05  | 35000.000000  |
| 50%   | 2014.000000 | 3.500000e+05  | 60000.000000  |
| 75%   | 2016.000000 | 6.000000e+05  | 90000.000000  |
| max   | 2020.000000 | 8.900000e+06  | 806599.000000 |

- **df.nunique():**  
This function shows the unique values in cells in each column.

```
[26]: df.nunique()
[26]:
```

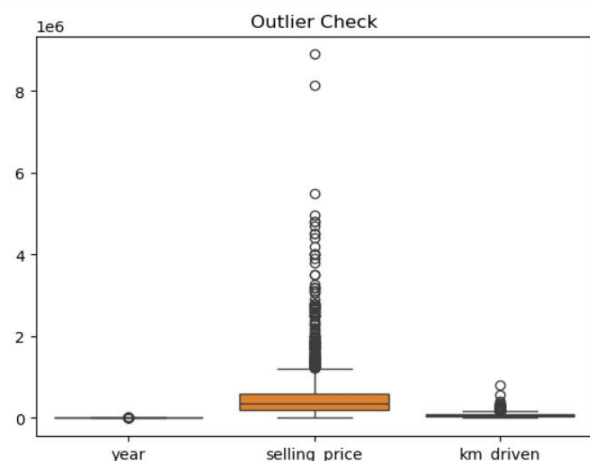
|               | name |
|---------------|------|
| year          | 1491 |
| selling_price | 27   |
| km_driven     | 445  |
| fuel          | 778  |
| seller_type   | 5    |
| transmission  | 3    |
| owner         | 2    |
| dtype: int64  | 5    |

- 
- **dat=dat.drop\_duplicates():** Delete the duplicated rows so we clean the table of any unnecessary cell.
  - **dat = dat.copy():**  
It severs any connection with the old data and makes you deal with a completely new and independent copy in memory.

- **sns.boxplot(...):**  
We tell the Seaborn library, "Draw me a box for this data." `data=dat[['year', 'selling_price', 'km_driven']]:` We specified only 3 columns (numeric data) to compare (year, selling price, and kilometers traveled).

```
[441]: import seaborn as sns
import matplotlib.pyplot as plt

sns.boxplot(data=dat[['year', 'selling_price', 'km_driven']])
plt.title("Outlier Check")
plt.show()
```



- `plt.title("Outlier Check"):`

We give the drawing a title:  
"Checking for Outliers".

- `Q=dat['selling_price'].quantile(0.25)`
- `Q3 =`  
`dat['selling_price'].quantile(0.75)`

This function successfully show identified 170 outliers in the `selling_price` column. These are the specific cars that are causing the "dots" you saw in the boxplot.

And there is a practical application and its analysis:-

**lower\_bound (minimum limit):** Any price below this is considered "abnormal" (low).

**upper\_bound (maximum limit):** Any price above this is considered "abnormal" (excessive).

The results show the following:

**Minimum: -400,000:**

This number is negative, meaning there are no cars with an "outlier" price in your data, as no car has a price below zero. So, you're safe in this respect.

**Maximum: 1,200,000:**

This is the magic number. It tells you that the "normal limit" for your most expensive car should be 1,200,000.

Any car with a price higher than 1,200,000 is considered an outlier.

- `dat = dat[(dat['selling_price'] >= lower_bound) & (dat['selling_price'] <= upper_bound)]`

```
[442]: Q1 = dat['selling_price'].quantile(0.25) # الربع الأول
      Q3 = dat['selling_price'].quantile(0.75) # الربع الثالث
      IQR = Q3 - Q1 # المدى بين الأرباع
      # عدد الانبعاثات
      # القيم التي تعتبر Outliers
      outliers = dat[(dat['selling_price'] < Q1 - 1.5*IQR) | (dat['selling_price'] > Q3 + 1.5*IQR)]
      print(outliers['selling_price'])
```

|      |         |
|------|---------|
| 12   | 1650000 |
| 29   | 1964999 |
| 30   | 1425000 |
| 35   | 1735000 |
| 36   | 1375000 |
| ...  | ...     |
| 4200 | 1250000 |
| 4204 | 1800000 |
| 4224 | 2500000 |
| 4228 | 1225000 |
| 4313 | 2800000 |

Name: selling\_price, Length: 170, dtype: int64

```
[443]: Q1 = dat['selling_price'].quantile(0.25) # الربع الأول
      Q3 = dat['selling_price'].quantile(0.75) # الربع الثالث
      IQR = Q3 - Q1 # المدى بين الأرباع

      lower_bound = Q1 - 1.5 * IQR
      upper_bound = Q3 + 1.5 * IQR

      print("الحد الأدنى:", lower_bound)
      print("الحد الأعلى:", upper_bound)
```

الحد الأدنى: -400000.0  
الحد الأعلى: 1200000.0

**Data filtering:** This line removes any car with an "abnormal" price (more than 1.2 million) to protect the model from potentially misleading figures.

**Prediction accuracy:** This step forces the program to learn only from reasonable and common prices, making your model's predictions more accurate and closer to reality

```
[444]: dat = dat[(dat['selling_price'] >= lower_bound) & (dat['selling_price'] <= upper_bound)]
dat
```

```
[444]:
```

|      | name                                | year | selling_price | km_driven | fuel   | seller_type | transmission | owner        |
|------|-------------------------------------|------|---------------|-----------|--------|-------------|--------------|--------------|
| 0    | Maruti 800 AC                       | 2007 | 60000         | 70000     | Petrol | Individual  | Manual       | First Owner  |
| 1    | Maruti Wagon R LXI Minor            | 2007 | 135000        | 50000     | Petrol | Individual  | Manual       | First Owner  |
| 2    | Hyundai Verna 1.6 SX                | 2012 | 600000        | 100000    | Diesel | Individual  | Manual       | First Owner  |
| 3    | Datsun RediGO T Option              | 2017 | 250000        | 46000     | Petrol | Individual  | Manual       | First Owner  |
| 4    | Honda Amaze VX i-DTEC               | 2014 | 450000        | 141000    | Diesel | Individual  | Manual       | Second Owner |
| ...  | ...                                 | ...  | ...           | ...       | ...    | ...         | ...          | ...          |
| 4335 | Hyundai i20 Magna 1.4 CRDi (Diesel) | 2014 | 409999        | 80000     | Diesel | Individual  | Manual       | Second Owner |
| 4336 | Hyundai i20 Magna 1.4 CRDi          | 2014 | 409999        | 80000     | Diesel | Individual  | Manual       | Second Owner |
| 4337 | Maruti 800 AC BSIII                 | 2009 | 110000        | 83000     | Petrol | Individual  | Manual       | Second Owner |
| 4338 | Hyundai Creta 1.6 CRDi SX Option    | 2016 | 865000        | 90000     | Diesel | Individual  | Manual       | First Owner  |
| 4339 | Renault KWID RXT                    | 2016 | 225000        | 40000     | Petrol | Individual  | Manual       | First Owner  |

3407 rows × 8 columns

- from sklearn.preprocessing import MinMaxScaler import pandas as pd  
scaler = MinMaxScaler()  
dat['engine\_size'] = dat['name'].str.extract(r'(\d+\.\d+)')  
dat['engine\_size'] = pd.to\_numeric(dat['engine\_size'], errors='coerce')  
dat['engine\_size'] = dat['engine\_size'].fillna(dat['engine\_size'].median())

**Line 1:** This line enters the car's name and extracts the engine size (e.g., 1.4 or 1.6) into a new column.

**Line 2:** This line converts these numbers from written text into actual numbers that the computer can calculate.

**Line 3:** If a car's name doesn't include a number, it enters the average number to avoid leaving empty spaces in the table.

- dat['engine\_size'] = scaler.fit\_transform(dat[['engine\_size']])  
dat['km\_driven'] = scaler.fit\_transform(dat[['km\_driven']])

**Normalization** of the numbers in the two columns (engine capacity and distance traveled) so that they all fall within one range (often between 0 and 1).

**Scaling:** The `fit\_transform` function was used to calculate and programmatically transform the values.

```
[19]: from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
dat['engine_size'] = dat['name'].str.extract(r'(\d+\.\d+)')
dat['engine_size'] = pd.to_numeric(dat['engine_size'], errors='coerce')
dat['engine_size'] = dat['engine_size'].fillna(dat['engine_size'].median())

dat
```

```
[19]:
```

|      | name                                | year | selling_price | km_driven | fuel   | seller_type | transmission | owner        | engine_size |
|------|-------------------------------------|------|---------------|-----------|--------|-------------|--------------|--------------|-------------|
| 0    | Maruti 800 AC                       | 2007 | 60000         | 70000     | Petrol | Individual  | Manual       | First Owner  | 1.4         |
| 1    | Maruti Wagon R LXI Minor            | 2007 | 135000        | 50000     | Petrol | Individual  | Manual       | First Owner  | 1.4         |
| 2    | Hyundai Verna 1.6 SX                | 2012 | 600000        | 100000    | Diesel | Individual  | Manual       | First Owner  | 1.6         |
| 3    | Datsun RediGO T Option              | 2017 | 250000        | 46000     | Petrol | Individual  | Manual       | First Owner  | 1.4         |
| 4    | Honda Amaze VX i-DTEC               | 2014 | 450000        | 141000    | Diesel | Individual  | Manual       | Second Owner | 1.4         |
| ...  | ...                                 | ...  | ...           | ...       | ...    | ...         | ...          | ...          | ...         |
| 4335 | Hyundai i20 Magna 1.4 CRDi (Diesel) | 2014 | 409999        | 80000     | Diesel | Individual  | Manual       | Second Owner | 1.4         |
| 4336 | Hyundai i20 Magna 1.4 CRDi          | 2014 | 409999        | 80000     | Diesel | Individual  | Manual       | Second Owner | 1.4         |
| 4337 | Maruti 800 AC BSIII                 | 2009 | 110000        | 83000     | Petrol | Individual  | Manual       | Second Owner | 1.4         |
| 4338 | Hyundai Creta 1.6 CRDi SX Option    | 2016 | 865000        | 90000     | Diesel | Individual  | Manual       | First Owner  | 1.6         |
| 4339 | Renault KWID RXT                    | 2016 | 225000        | 40000     | Petrol | Individual  | Manual       | First Owner  | 1.4         |

3407 rows × 9 columns

```
[20]: #normalize
dat['engine_size'] = scaler.fit_transform(dat[['engine_size']])
dat['km_driven'] = scaler.fit_transform(dat[['km_driven']])
dat
```

```
[20]:
```

|      | name                                | year | selling_price | km_driven | fuel   | seller_type | transmission | owner        | engine_size |
|------|-------------------------------------|------|---------------|-----------|--------|-------------|--------------|--------------|-------------|
| 0    | Maruti 800 AC                       | 2007 | 60000         | 0.086783  | Petrol | Individual  | Manual       | First Owner  | 0.181818    |
| 1    | Maruti Wagon R LXI Minor            | 2007 | 135000        | 0.061988  | Petrol | Individual  | Manual       | First Owner  | 0.181818    |
| 2    | Hyundai Verna 1.6 SX                | 2012 | 600000        | 0.123976  | Diesel | Individual  | Manual       | First Owner  | 0.272727    |
| 3    | Datsun RediGO T Option              | 2017 | 250000        | 0.057028  | Petrol | Individual  | Manual       | First Owner  | 0.181818    |
| 4    | Honda Amaze VX i-DTEC               | 2014 | 450000        | 0.174807  | Diesel | Individual  | Manual       | Second Owner | 0.181818    |
| ...  | ...                                 | ...  | ...           | ...       | ...    | ...         | ...          | ...          | ...         |
| 4335 | Hyundai i20 Magna 1.4 CRDi (Diesel) | 2014 | 409999        | 0.099181  | Diesel | Individual  | Manual       | Second Owner | 0.181818    |
| 4336 | Hyundai i20 Magna 1.4 CRDi          | 2014 | 409999        | 0.099181  | Diesel | Individual  | Manual       | Second Owner | 0.181818    |
| 4337 | Maruti 800 AC BSIII                 | 2009 | 110000        | 0.102900  | Petrol | Individual  | Manual       | Second Owner | 0.181818    |
| 4338 | Hyundai Creta 1.6 CRDi SX Option    | 2016 | 865000        | 0.111579  | Diesel | Individual  | Manual       | First Owner  | 0.272727    |
| 4339 | Renault KWID RXT                    | 2016 | 225000        | 0.049590  | Petrol | Individual  | Manual       | First Owner  | 0.181818    |

3407 rows × 9 columns



**Application:** Large distances and engine capacity numbers were converted to decimal fractions (e.g., 0.086).

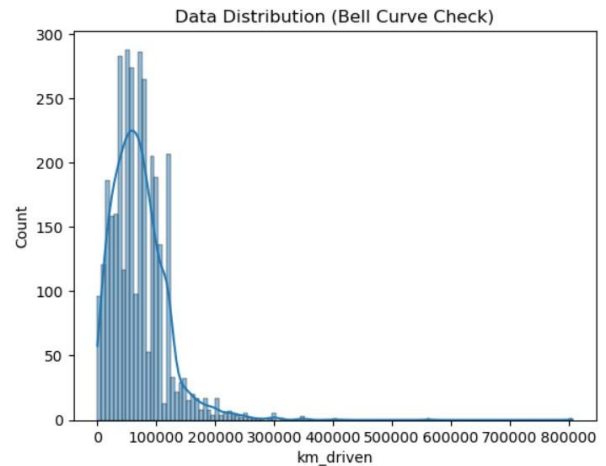
**Result:** This procedure prevents large numbers from negatively impacting the accuracy of the AI model, resulting in faster and more precise learning.

- **sns.histplot(dat['km\_driven'], kde=True)**

**The bar chart (Histogram):** This divides the distance traveled by the vehicles into groups, telling you, "How many vehicles traveled from 0 to 50,000 kilometers? And how many vehicles traveled from 50,000 to 100,000 kilometers?" and so on.

**The blue line (KDE):** This is called a "curve," and it shows you the general distribution of the data so you can see if it follows a "bell" pattern (normal distribution) or what the pattern is like.

```
[445]: sns.histplot(dat['km_driven'], kde=True)
plt.title("Data Distribution (Bell Curve Check)")
plt.show()
```



- **Scatter Plot:**

1. It consists of points. Each point represents one of the cars in your table. The position of the point tells you two things simultaneously:

Its position on the X-axis tells you how many kilometers this car has traveled (km\_driven).

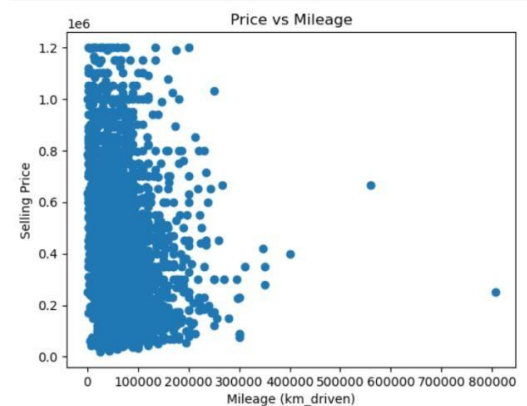
Its position on the Y-axis tells you how much this car costs (selling\_price).

2. It explains the relationship between the year of manufacture and the selling price.

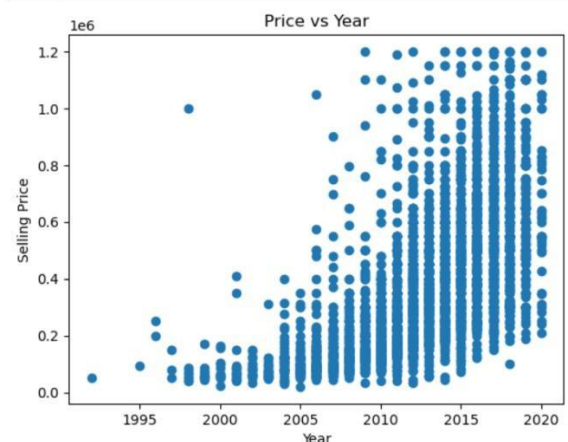
**A clear direct correlation:** we'll notice that the points start to "rise" as you move to the right in the years. This simply means that newer cars (2015 to 2020 models) are much more expensive than older models.

The large number of points is between 2010 and 2020, which tells that most of the cars in table are relatively recent models.

```
[446]: import numpy as np
plt.scatter(dat['km_driven'], dat['selling_price'])
plt.xlabel("Mileage (km_driven)")
plt.ylabel("Selling Price")
plt.title("Price vs Mileage")
plt.show()
```



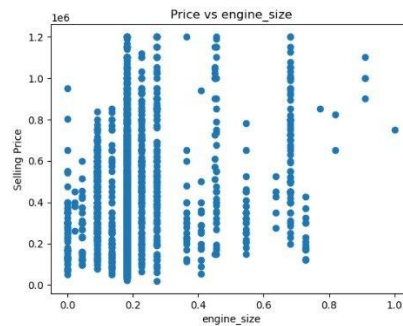
```
[461]: plt.scatter(dat['year'], dat['selling_price'])
plt.xlabel("Year")
plt.ylabel("Selling Price")
plt.title("Price vs Year")
plt.show()
```



- `plt.scatter(dat['engine_size'], dat['selling_price'])`  
`plt.xlabel("engine_size")`  
`plt.ylabel("Selling Price")`  
`plt.title("Price vs engine_size")`  
`plt.show()`

The vertical shape of the points: The points appear as vertical columns because the engine displacement values are fixed and originally defined (such as 1200cc or 1600cc) and are repeated with varying prices.

```
[24]: plt.scatter(dat['engine_size'], dat['selling_price'])
plt.xlabel("engine_size")
plt.ylabel("Selling Price")
plt.title("Price vs engine_size")
plt.show()
```



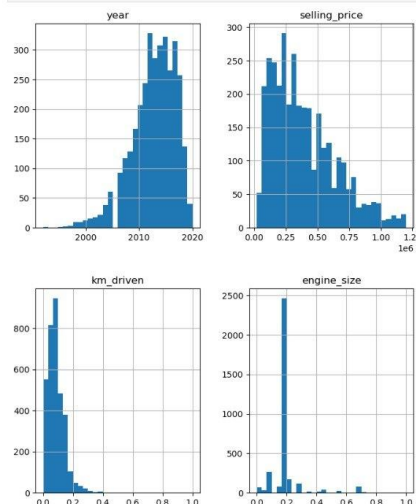
Price distribution: Most cars are concentrated in the price range below 600,000 ( $0.6 \times 10^6$ ), with very few reaching 1.2 million.

Nature of the relationship: There is no explicit linear relationship (a straight upward line), but it appears that engines with values close to 0.2 and 0.7 are the most common and have the greatest price variation.

- **`dat.hist()` :**

This code plots the frequency distribution of all the numerical columns together, clearly showing where the data is concentrated:

```
[25]: # Plots a histogram for every number column in your table
dat.hist(bins=30, figsize=(8, 10))
plt.show()
```



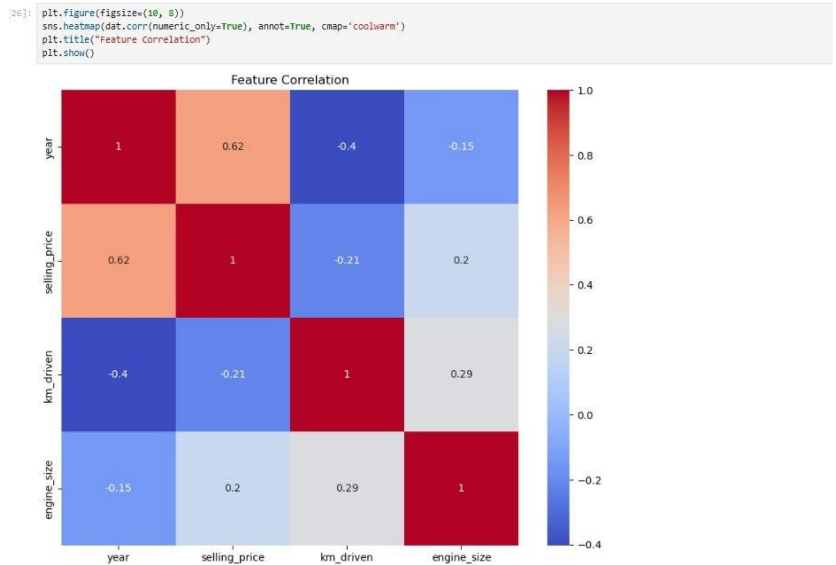
Year of Manufacture (year): The curve shows that the majority of cars are relatively new, concentrated between 2010 and 2020, with a slight decrease in the number of cars around 2007.

Selling Price (selling\_price): The graph shows that prices are heavily concentrated in the low and mid-range segments, stopping precisely at the 1.25 million mark, confirming the success of the outliers cleanup process.

Mileage and Engine Size (km\_driven & engine\_size): The graphs confirm the success of the normalization step; the values on the horizontal axis of both columns are now precisely between 0 and 1..

## □ Heatmap:

The **dat.corr()** code measures the strength of the relationship between the columns numerically. If the number is positive and close to 1, the relationship is direct and strong (e.g., year and price, where it's 0.62), and if it's negative, the relationship is inverse (e.g., mileage and price, where it's -0.21, and mileage and year, where it's -0.4).



The color gradient from red (positive) to blue (negative) tells you at a glance which factor has the greatest impact on the car's price.

- **encoder = LabelEncoder()**  
**dat.loc[:, 'owner\_num'] = encoder.fit\_transform(dat['owner'])**
- **encoder = LabelEncoder()**  
**dat.loc[:, 'fuel\_num'] = encoder.fit\_transform(dat['fuel'])**
- **encoder = LabelEncoder()**  
**dat.loc[:, 'seller\_type\_num'] = encoder.fit\_transform(dat['seller\_type'])**

These codes use a tool called **LabelEncoder** to convert textual data (categorical data) into numbers, because AI models don't understand text and need everything to be numerical.

How it works: The tool iterates through the column and assigns each unique word a number (for example, in the Type column: Individual is categorized as 0, Dealer as 1, and so on).

The application in the image: This process was applied to three columns, and new parallel columns were created with numbers:

```
[27]: from sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()
dat.loc[:, 'owner_num'] = encoder.fit_transform(dat['owner'])

[28]: from sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()
dat.loc[:, 'fuel_num'] = encoder.fit_transform(dat['fuel'])

[29]: from sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()
dat.loc[:, 'seller_type_num'] = encoder.fit_transform(dat['seller_type'])
```

1. The column for the number of previous owners (owner) was converted to owner\_num.
2. The column for the fuel type (fuel) was converted to fuel\_num.
3. The column for the seller type (seller\_type) was converted to seller\_type\_num.



- **from sklearn.preprocessing import LabelEncoder**  
**encoder = LabelEncoder()**  
**dat.loc[:, 'transmission\_num'] = encoder.fit\_transform(dat['transmission'])**  
**dat**

This code is a continuation of the process of converting text data to numbers (Label Encoding), and here it's applied to the transmission type (gearbox).

It converted the text words in the transmission column (such as Manual or Automatic) into unique numerical values within a completely new column called transmission\_num.

The last line, ``dat``, displays the complete table after the modification so you can preview the result.

```
[39]: from sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()
dat.loc[:, 'transmission_num'] = encoder.fit_transform(dat['transmission'])
dat
```

```
[39]:
```

|      | name                                | year | selling price | km driven | fuel   | seller type | transmission | owner        | engine size | owner num | fuel num | seller type num | transmission num |
|------|-------------------------------------|------|---------------|-----------|--------|-------------|--------------|--------------|-------------|-----------|----------|-----------------|------------------|
| 0    | Maruti 800 AC                       | 2007 | 60000         | 0.086783  | Petrol | Individual  | Manual       | First Owner  | 0.181818    | 0         | 4        | 1               | 1                |
| 1    | Maruti Wagon R LXI Minor            | 2007 | 135000        | 0.061988  | Petrol | Individual  | Manual       | First Owner  | 0.181818    | 0         | 4        | 1               | 1                |
| 2    | Hyundai Verna 1.6 SX                | 2012 | 600000        | 0.123976  | Diesel | Individual  | Manual       | First Owner  | 0.272727    | 0         | 1        | 1               | 1                |
| 3    | Datsun RediGO T Option              | 2017 | 250000        | 0.057028  | Petrol | Individual  | Manual       | First Owner  | 0.181818    | 0         | 4        | 1               | 1                |
| 4    | Honda Amaze VX i-DTEC               | 2014 | 450000        | 0.174807  | Diesel | Individual  | Manual       | Second Owner | 0.181818    | 2         | 1        | 1               | 1                |
| ...  | ...                                 | ...  | ...           | ...       | ...    | ...         | ...          | ...          | ...         | ...       | ...      | ...             | ...              |
| 4335 | Hyundai i20 Magna 1.4 CRDi (Diesel) | 2014 | 409999        | 0.099181  | Diesel | Individual  | Manual       | Second Owner | 0.181818    | 2         | 1        | 1               | 1                |
| 4336 | Hyundai i20 Magna 1.4 CRDi          | 2014 | 409999        | 0.099181  | Diesel | Individual  | Manual       | Second Owner | 0.181818    | 2         | 1        | 1               | 1                |
| 4337 | Maruti 800 AC BSIII                 | 2009 | 110000        | 0.102900  | Petrol | Individual  | Manual       | Second Owner | 0.181818    | 2         | 4        | 1               | 1                |
| 4338 | Hyundai Creta 1.6 CRDi SX Option    | 2016 | 865000        | 0.111579  | Diesel | Individual  | Manual       | First Owner  | 0.272727    | 0         | 1        | 1               | 1                |
| 4339 | Renault KWID RXT                    | 2016 | 225000        | 0.049590  | Petrol | Individual  | Manual       | First Owner  | 0.181818    | 0         | 4        | 1               | 1                |

3407 rows x 13 columns

We notice that the new numerical columns created in the previous steps appear entirely on the right side of the table (**owner\_num**, **fuel\_num**, **seller\_type\_num**, **transmission\_num**).

For example, the word "Manual" is assigned the number 1, "First Owner" is assigned the number 0, and "Second Owner" is assigned the number 2.

The number of columns in the table has officially increased to 13 instead of the initial 9.

• **dat = dat.drop(columns=['name', 'brand'])**  
**dat**

The `drop(columns=[...])` function permanently removes the columns enclosed within the brackets from the table. Here, the `name` and `brand` columns have been deleted.

Reason: Since we encoded this text data in the previous steps, leaving the original text columns would cause an error when the AI model inputs the data, as the model only understands numeric code.

```
[33]: dat = dat.drop(columns=['name', 'brand'])
dat
```

```
[33]:
```

|      | year | selling_price | km_driven | engine_size | owner_num | fuel_num | seller_type_num | transmission_num | brand_num |
|------|------|---------------|-----------|-------------|-----------|----------|-----------------|------------------|-----------|
| 0    | 2007 | 60000         | 0.086783  | 0.181818    | 0         | 4        | 1               | 1                | 12        |
| 1    | 2007 | 135000        | 0.061988  | 0.181818    | 0         | 4        | 1               | 1                | 12        |
| 2    | 2012 | 600000        | 0.123976  | 0.272727    | 0         | 1        | 1               | 1                | 10        |
| 3    | 2017 | 250000        | 0.057028  | 0.181818    | 0         | 4        | 1               | 1                | 5         |
| 4    | 2014 | 450000        | 0.174807  | 0.181818    | 2         | 1        | 1               | 1                | 9         |
| ...  | ...  | ...           | ...       | ...         | ...       | ...      | ...             | ...              | ...       |
| 4335 | 2014 | 409999        | 0.099181  | 0.181818    | 2         | 1        | 1               | 1                | 10        |
| 4336 | 2014 | 409999        | 0.099181  | 0.181818    | 2         | 1        | 1               | 1                | 10        |
| 4337 | 2009 | 110000        | 0.102900  | 0.181818    | 2         | 4        | 1               | 1                | 12        |
| 4338 | 2016 | 865000        | 0.111579  | 0.272727    | 0         | 1        | 1               | 1                | 10        |
| 4339 | 2016 | 225000        | 0.049590  | 0.181818    | 0         | 4        | 1               | 1                | 17        |

3407 rows x 9 columns

```
[34]: import datetime
current_year = datetime.datetime.now().year
dat['car_age'] = current_year - dat['year']
dat = dat.drop(columns=['year'])

print( dat.columns.tolist())

['selling_price', 'km_driven', 'engine_size', 'owner_num', 'fuel_num', 'seller_type_num', 'transmission_num', 'brand_num', 'car_age']
```

• **import datetime**  
**current\_year = datetime.datetime.now().year**  
**dat['car\_age'] = current\_year - dat['year']**  
**at = dat.drop(columns=['year'])**  
**print(dat.columns.tolist())**

Lines 1 and 2: Invoke the `datetime` library, retrieve the current year, and store it in a variable named `current_year`.

Line 3: Create a new column in the table named `car_age`. Its value is calculated by subtracting the car's manufacturing year from the current year.

```
[33]: dat = dat.drop(columns=['name', 'brand'])
dat
```

```
[33]:
```

|      | year | selling_price | km_driven | engine_size | owner_num | fuel_num | seller_type_num | transmission_num | brand_num |
|------|------|---------------|-----------|-------------|-----------|----------|-----------------|------------------|-----------|
| 0    | 2007 | 60000         | 0.086783  | 0.181818    | 0         | 4        | 1               | 1                | 12        |
| 1    | 2007 | 135000        | 0.061988  | 0.181818    | 0         | 4        | 1               | 1                | 12        |
| 2    | 2012 | 600000        | 0.123976  | 0.272727    | 0         | 1        | 1               | 1                | 10        |
| 3    | 2017 | 250000        | 0.057028  | 0.181818    | 0         | 4        | 1               | 1                | 5         |
| 4    | 2014 | 450000        | 0.174807  | 0.181818    | 2         | 1        | 1               | 1                | 9         |
| ...  | ...  | ...           | ...       | ...         | ...       | ...      | ...             | ...              | ...       |
| 4335 | 2014 | 409999        | 0.099181  | 0.181818    | 2         | 1        | 1               | 1                | 10        |
| 4336 | 2014 | 409999        | 0.099181  | 0.181818    | 2         | 1        | 1               | 1                | 10        |
| 4337 | 2009 | 110000        | 0.102900  | 0.181818    | 2         | 4        | 1               | 1                | 12        |
| 4338 | 2016 | 865000        | 0.111579  | 0.272727    | 0         | 1        | 1               | 1                | 10        |
| 4339 | 2016 | 225000        | 0.049590  | 0.181818    | 0         | 4        | 1               | 1                | 17        |

3407 rows x 9 columns

```
[34]: import datetime
current_year = datetime.datetime.now().year
dat['car_age'] = current_year - dat['year']
dat = dat.drop(columns=['year'])

print( dat.columns.tolist())

['selling_price', 'km_driven', 'engine_size', 'owner_num', 'fuel_num', 'seller_type_num', 'transmission_num', 'brand_num', 'car_age']
```

(The "age" feature is more accurate and easier for the AI model to learn than the "manufacturing year" as a simple numerical value.)

Line 4: After successfully retrieving the car's age, the original manufacturing year column is no longer needed, so it is dropped using the drop function.

Line 5: Prints the names of all remaining columns in the table as a list to ensure the table is completely clean and contains only numerical data.

• **X = dat.drop(columns=['selling\_price'])**  
**Y = dat['selling\_price']**

This code tells the computer what the "data" (X) is that it will learn from, and what the "predictive answer or target" (Y) is that we are trying to predict (which is the price).

```
[36]: #splitting
X = dat.drop(columns=['selling_price'])
Y = dat['selling_price']

from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, random_state=42
)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)

X_train shape: (2725, 8)
X_test shape: (682, 8)
```

The splitting function `train\_test\_split` in the following lines depends entirely on the presence of X and Y ready to be split into 80% and 20%.

Training Step: In the next step, the AI model needs to hear `model.fit(X\_train, Y\_train)`. Without separating X and Y, the model cannot learn or work.

• **model\_rf.fit(X\_train, Y\_train)**

Creating the algorithmic model

**(RandomForestRegressor):**

This involves building 200 decision trees to combine their predictions and arrive

at the most accurate price, with a maximum branching limit to ensure the tree remains uncomplicated.

```
[37]: from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_absolute_error

model_rf = RandomForestRegressor(n_estimators=200, max_depth=15, min_samples_split=5, random_state=42)
model_rf.fit(X_train, Y_train)
predicted_rf = model_rf.predict(X_test)

print(f"R2 Score: {r2_score(Y_test, predicted_rf):.2f}")
print(f"MAE: {mean_absolute_error(Y_test, predicted_rf):.2f}")

R2 Score: 0.70
MAE: 94598.20
```

Training and Prediction (**Fit & Predict**): The model was trained on 80% of the data to understand the relationship between car specifications and prices, and then its ability to predict the prices of the remaining 20% was tested.

Evaluating the results and success indicators: Prediction accuracy ( $R^2$  \ Score = 0.70): This means the model was able to understand and explain 70% of the reasons for the differences in car prices.

Error percentage (MAE = 94,598): This means the model's predictions differ from the actual car price by approximately 94,600 (over or under).

```
•sorted_order = actual_50.argsort()
actual_sorted = actual_50[sorted_order]
predicted_sorted = predicted_50[sorted_order]
```

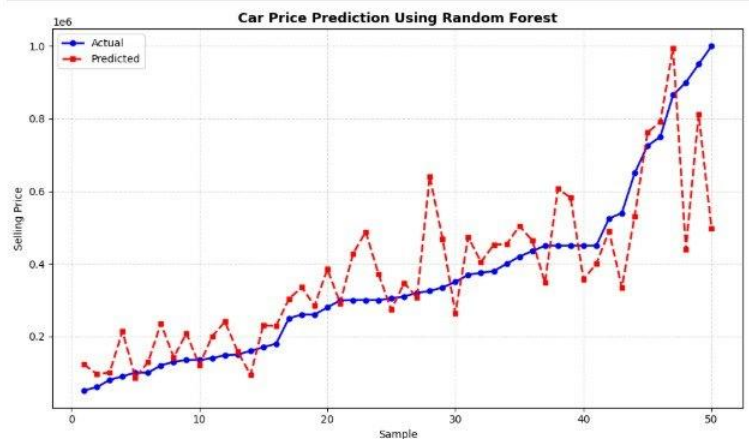
This code aims to provide a direct visual comparison between the actual prices of cars and the prices predicted by the model for only 50 samples, arranged in ascending order for easier viewing.

```
18]: actual_50 = Y_test.values[:50]
predicted_50 = predicted_rf[:50]

sorted_order = actual_50.argsort()
actual_sorted = actual_50[sorted_order]
predicted_sorted = predicted_50[sorted_order]

plt.figure(facecolor='white', figsize=(18, 6))
plt.plot(range(1, len(actual_sorted)+1), actual_sorted, 'o-', color='blue', label='Actual', linewidth=2, markersize=5)
plt.plot(range(1, len(predicted_sorted)+1), predicted_sorted, 's--', color='red', label='Predicted', linewidth=2, markersize=5)
plt.title("Car Price Prediction Using Random Forest", fontsize=13, fontweight='bold')
plt.xlabel("Sample")
plt.ylabel("Selling Price")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```

1. Data preparation and arrangement: The code takes the first 50 cars from the test group and arranges them from lowest to highest price, displaying the actual prices as a rising, organized line.
2. The solid blue line (Actual): represents the actual prices of the cars in the table.
3. The dashed red line (Predicted): represents the AI model's predictions for the same cars.



We observe that the red line accurately follows the overall rising trajectory of the blue line. This convergence and similarity at many points visually reflects the model's accuracy ( $R^2 = 0.70$ ), demonstrating its high ability to predict prices very close to reality, with some natural fluctuations (error) present in certain samples.